

Bachelor Thesis

Machine-Learning-Based Receivers for Low-Resolution Receiver Frontends

Makaryo Latif

Supervisor:

Dr.-Ing. Sebastian Stern

at the

**Institute of Communications Engineering
Ulm University**

Prof. Dr.-Ing. Robert Fischer

31 July 2025

Declaration

I confirm that the presented Bachelor thesis is an original work done independently without the use of any inadmissible outside help other than the indicated sources. I also certify that this work has not been submitted to any other examination board in the same or similar form.

Ulm, 31 July 2025

Makaryo Latif

Address: 142B Toman Bai, El-Zeitoun, Cairo, Egypt

Email: makaryo.latif@uni-ulm.de

Contents

Abstract	iv
List of Symbols	v
List of Abbreviations	vii
1 Introduction	1
2 System Model	3
2.1 Massive MIMO Uplink Scenario	3
2.2 Channel Model and Spatial Power Space Profile (PSP)	3
2.3 Differential Modulation (DPSK Encoding)	4
2.4 Channel Output and Noise Model	5
2.5 Multi-User Received Signal and Interference Modeling	5
2.6 Low-Resolution Quantization	6
2.7 Correlation Matrix Construction and Feature Extraction	7
2.8 Label Preparation	7
3 Conventional Non-Coherent Detection Methods	9
3.1 Decision-Feedback Differential Detection (DFDD)	9
3.2 Multiple-Symbol Differential Detection (MSDD)	10
4 Machine Learning Background and Tools	12
4.1 Machine Learning Classification for Non-Coherent Detection	12
4.1.1 Fundamentals of Classification	12

4.1.2 Neural Network Architecture	13
4.1.3 Activation Functions	13
4.1.4 Loss Functions and Optimization	13
4.1.5 Training Procedure	14
4.1.6 Evaluation Metrics	14
4.1.7 Types of Machine Learning and Motivation for Classification	14
4.2 Tools and Libraries for Model Training	15
5 Neural Network Architecture for Non-Coherent Detection	17
5.1 Block Prediction Approach	17
5.1.1 Output Structure and Label Encoding	17
5.1.2 Network Structure	18
5.1.3 Loss Function and Optimization	19
5.1.4 Limitations and Motivation for a New Architecture	19
5.2 Per-Symbol Parallel Prediction	20
5.2.1 Output Structure and Label Encoding	20
5.2.2 Network Architecture	21
5.2.3 Loss Function and Optimization	23
5.2.4 Input Features and Consistency Across Architectures	24
5.2.5 Advantages of Per-Symbol Architecture	24
5.3 Data Generation Techniques	25
5.3.1 Pre-Generated Dataset (Offline Sequence Generation)	25
5.3.2 On-the-Fly Generation (Sequential Data Generation During Training)	25
5.3.3 Summary of Trade-Offs	26
5.4 Quantized Data and Quantization Using Hyperbolic Tangent	26
5.4.1 1-Bit Quantization	27
5.4.2 Quantization Using Hyperbolic Tangent	27
5.4.3 Integration into Learning-Based Detection	28
5.4.4 Application and Justification	28

6	SER Performance Evaluation	30
6.1	Simulation Environment and Model Details	31
6.2	SER Performance Graphs and Results	31
6.2.1	Single User – Non-Quantized	31
6.2.2	Single User – 1-bit Quantized Training	33
6.2.3	Single User – Trained with Hyperbolic Tangent Quantizer	37
6.2.4	Multi-User Scenario – 1-bit Quantized Training	40
6.3	Conclusion	41
7	Summary and Outlook	43
7.1	Summary	43
7.2	Outlook	44
	References	45

Abstract

In this thesis, the problem of non-coherent detection in massive MIMO (MaMIMO) uplink systems with low-resolution (1-bit) quantized data is addressed. A multi-user scenario is considered, where each user employs differential modulation, and the base station operates without explicit channel state information.

The aim of this thesis is to investigate the effectiveness of deep learning-based detection architectures under severe quantization constraints. Two different network structures are considered: one that predicts entire symbol blocks jointly, and another that processes each symbol in parallel. Multiple data generation strategies are explored, including pre-training generation on fixed datasets and on-the-fly generation during training. Moreover, quantization techniques using the hyperbolic tangent function with adjustable sharpness are examined.

It is shown that the deep learning-based approach can achieve competitive or better symbol error rate (SER) performance compared to traditional decision-feedback differential detection (DFDD), particularly in lower SNR ranges and multi-user environments. The results further highlight the limitations of fixed quantization and emphasize the advantages of scalable architectures and pre-training data generation methods.

The results provide a strong foundation for further research in learning-based non-coherent detection, especially in hardware-limited communication systems. Future work may focus on advanced neural architectures, adaptive quantization mechanisms, and generalization across varying channel models.

List of Symbols

Symbol	Meaning
N_{rx}	Number of receive antennas
N_{u}	Number of single antenna users
N_{s}	Number of symbols in a single information block
$\mathbf{Y}$	Received signal
$\mathbf{H}$	Channel
$\mathbf{B}$	Matrix of DPSK-encoded symbols for all users
$\mathbf{N}$	Matrix of additive white Gaussian noise
u	User
r	Receiver antenna
$\mathbf{h}_u$	Channel vector for each user
$\mathbf{p}_u$	Power space profile vector for each user
$\mathbf{g}_u$	Standard complex Gaussian vector
$\circ$	Element wise multiplication
c_u	User peak antenna
M	Cardinality of constellation
a_u	Original information symbols
b_u	DPSK-encoded symbols
k	Time step (symbol index)
j	Imaginary unit
σ_{n}^2	Noise variance
Y^{q}	Quantized signal
$\Re$	Real part

Symbol	Meaning
$\Im$	Imaginary part
ζ	Channel spread parameter
$\mathcal{CN}$	Complex normal distribution
$\mathbf{Z}$	Correlation matrix
$\arg$	Argument
$\hat{k}_n$	Optimized detection order index
$ \cdot $	Absolute value
Δ	Phase quantization error
mod	Modulo operation
y	True probability
$\hat{y}$	Predicted probability
$\log$	Logarithmic function
C	Number of classes
L	Loss
$\tanh$	Hyperbolic tangent function
α	Soft quantization scaler
N_{samples}	Number of training samples
N_{total}	Total number of symbols across all symbols and time steps
Y_{n}	Noisy signal
$\lfloor \cdot \rfloor$	Floor function
E_{s}	Energy per symbol
N_0	Noise power spectral density
diag	Diagonal matrix
$\mathbf{x}$	Feature vector
$\text{vec}_{\text{upper}}$	Extract upper triangle

List of Abbreviations

Abbr.	Meaning
MIMO	Multiple Input Multiple Output
MaMIMO	Massive Multiple Input Multiple Output
SIMO	Single Input Multiple Output
CSI	Channel State Information
DPSK	Differential Phase-Shift Keying
DD	Differential Detection
DFDD	Decision Feedback Differential Detection
MSDD	Multiple Symbol Differential Detection
MSDSD	Multiple Symbol Differential Sphere Decoding
ML	Machine Learning
DL	Deep Learning
SER	Symbol Error Rate
AWGN	Additive White Gaussian Noise
PSP	Power Space Profile
SIC	Successive Interference Cancellation
MPSK	M-ary Phase Shift Keying
SGD	Stochastic Gradient Descent
GPU	Graphics Processing Unit
CPU	Central Processing Unit
ReLU	Rectified Linear Unit
ADC	Analog to Digital Converter

Chapter 1

Introduction

Massive multiple-input multiple-output (MIMO) systems have become a key technology in modern wireless communications. By using a large number of antennas at the base station, Massive MIMO can serve multiple users simultaneously and significantly improve spectral and energy efficiency. However, this comes at the cost of increased signal processing complexity, especially in scenarios with many users and when non-coherent transmission is used.

In coherent systems, the receiver requires explicit channel state information (CSI), which is costly and challenging to acquire in Massive MIMO scenarios due to the high-dimensional channels and rapid variations. As a result, non-coherent schemes are often preferred, which avoid explicit channel estimation and instead rely on differential modulation techniques such as differential phase-shift keying (DPSK). While these approaches simplify receiver design, they introduce challenges for reliable detection, particularly when dealing with multiple users and massive arrays.

An additional challenge arises when considering low-resolution data. In practical systems, signals may be quantized or coarsely processed due to hardware constraints or to reduce implementation complexity. However, working with low-resolution (quantized) data severely complicates detection because much of the fine-grained signal information is lost. Currently, there is no optimal detection scheme for such scenarios, making this an open and important problem.

Traditionally, methods such as decision feedback differential detection (DFDD) and multiple symbol differential detection (MSDD) have been used for non-coherent detection. DFDD is simpler but generally performs worse than MSDD, which jointly processes a block of symbols for improved accuracy. While MSDD provides better performance, it still struggles under massive MIMO conditions with strong interference and quantization effects.

Recently, machine learning (ML) based approaches, especially deep learning (DL), have attracted attention as powerful tools to improve detection performance. Instead of relying on strict analytical models, DL-based detectors can learn directly from data and thus potentially handle the complex and nonlinear effects introduced by massive antenna arrays and low-resolution data. In particular, prior research from [MMF21] has shown that a DL-based MSDD architecture can significantly outperform DFDD and achieve performance close to that of multiple-symbol differential sphere decoding (MSDSD) in terms of symbol error rate (SER) with non-quantized signals.

The main objective of this thesis is to investigate the use of ML-based receivers for massive MIMO systems using low-resolution samples. The aim is to design and analyze a deep learning detection architecture capable of operating under quantized data constraints, and evaluate their performance compared to conventional DFDD schemes. This work highlights the behavior of deep learning under these challenging conditions and quantifies the resulting SER performance.

This thesis is organized as follows. Chapter 2 introduces the system model, detailing the massive MIMO setup, channel characteristics, and signal processing steps including quantization. Chapter 3 presents the theoretical background for conventional non-coherent detection schemes. Chapter 4 explores machine learning basics with a focus on classification tasks relevant to receiver design and the tools used for implementing the DL structure. Chapter 5 describes the receiver-side architecture, explaining the neural network models and their evolution throughout this work, including details of the different architecture versions, data generation methods and quantization techniques. Chapter 6 provides the performance evaluation, comparing the deep learning approach to DFDD. Finally, Chapter 7 summarizes the main findings and outlines possible future research directions.

Chapter 2

System Model

This chapter describes the system setup and mathematical signal model used in this thesis. The communication system model — a multi-user massive multiple-input multiple-output (MaMIMO) uplink scenario employing non-coherent differential modulation — is adopted from [MMF21]. Building on this established framework, this thesis extends the model by incorporating low-resolution quantized data at the receiver, representing a novel aspect of the work. This detailed modeling lays the foundation for designing and analyzing machine-learning-based detection approaches.

2.1 Massive MIMO Uplink Scenario

The study considers a massive MIMO uplink system consisting of a base station equipped with N_{rx} receive antennas and N_{u} single-antenna users, where $N_{\text{rx}} \gg N_{\text{u}}$. Over a block of N_{s} symbol intervals, the received signal matrix $\mathbf{Y} \in \mathbb{C}^{N_{\text{rx}} \times N_{\text{s}}}$ can be expressed as

$$\mathbf{Y} = \mathbf{H}\mathbf{B} + \mathbf{N}, \quad (2.1)$$

where $\mathbf{H} \in \mathbb{C}^{N_{\text{rx}} \times N_{\text{u}}}$ denotes the channel matrix, $\mathbf{B} \in \mathbb{C}^{N_{\text{u}} \times N_{\text{s}}}$ is the matrix of transmitted DPSK-encoded symbols for all users, and $\mathbf{N} \in \mathbb{C}^{N_{\text{rx}} \times N_{\text{s}}}$ represents additive white Gaussian noise (AWGN).

2.2 Channel Model and Spatial Power Space Profile (PSP)

The channel matrix $\mathbf{H}$ captures flat Rayleigh fading and incorporates user-specific spatial power space profiles (PSPs). This channel and PSP model follows the approach in

[MMF21]. For user u , the channel vector $\mathbf{h}_u \in \mathbb{C}^{N_{rx}}$ is generated as

$$\mathbf{h}_u = \sqrt{\mathbf{p}_u} \circ \mathbf{g}_u, \quad (2.2)$$

where $\mathbf{g}_u \sim \mathcal{CN}(\mathbf{0}, \mathbf{I})$ is a standard complex Gaussian vector, $\mathbf{p}_u$ is the PSP vector for user u , and $\circ$ denotes element-wise multiplication.

The PSP vector $\mathbf{p}_u$ is defined using a Gaussian-shaped distribution centered at antenna index c_u which is the user peak antenna with spread parameter ζ

$$p_{r,u} = \frac{\exp\left(-\frac{(r-c_u)^2}{2\zeta^2}\right)}{\sum_{r=1}^{N_{rx}} \exp\left(-\frac{(r-c_u)^2}{2\zeta^2}\right)}, \quad (2.3)$$

where r is the antenna index. This PSP approximates the angular power profile of each user and is a practical alternative to full instantaneous channel state information (CSI), which is costly to acquire. Figure 2.1 shows the massive MIMO uplink scenario considered in this work.

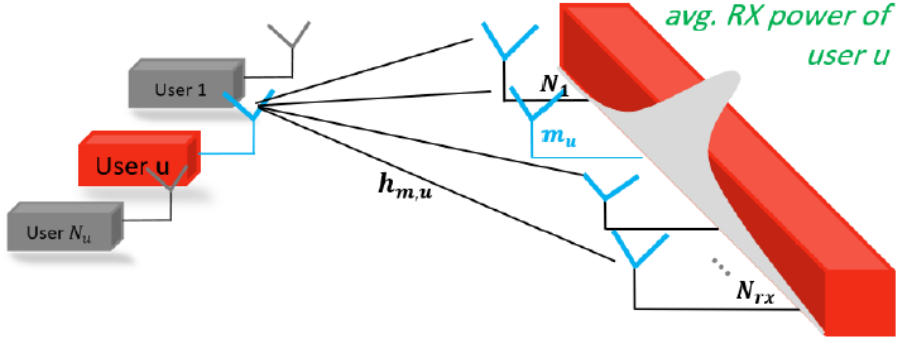


Figure 2.1: Illustration of the massive MIMO uplink scenario with multiple single-antenna users and a base station with many antennas. Taken from [MMF21].

2.3 Differential Modulation (DPSK Encoding)

To avoid the need for explicit carrier phase synchronization, M -ary differential phase shift keying (M-DPSK) modulation is employed. The concept of DPSK modulation follows the explanation given in [FH23]. For each user u , an information sequence $\{a_u[k]\}$ with $a_u[k] \in \{0, 1, \dots, M-1\}$ is generated. The differentially encoded symbols are then recursively defined as

$$b_u[k] = b_u[k-1] \exp\left(j \frac{2\pi}{M} a_u[k]\right), \quad (2.4)$$

with an initial reference symbol $b_u[0] = 1$. This approach allows non-coherent detection by encoding information in the phase differences rather than the absolute phase.

2.4 Channel Output and Noise Model

The transmitted symbol matrix $\mathbf{B}$ collects the symbols from all users over N_s time slots. After propagation through the channel $\mathbf{H}$, the received signal $\mathbf{Y}$ is corrupted by noise $\mathbf{N}$, whose entries follow

$$N_{r,k} \sim \mathcal{CN}(0, \sigma_n^2). \quad (2.5)$$

Here, r and k denote the receive antenna and time index, respectively.

Due to unit-energy PSK modulation, the average transmitted symbol energy per user is $\mathbb{E}[|b_u[k]|^2] = 1$, where $\mathbf{b}_u[k] = [b_{u,1}[k], b_{u,2}[k], \dots, b_{u,N_s}[k]]$ denotes the transmitted symbol vector of user u at time k . The signal-to-noise ratio (SNR) is defined as the ratio between energy per symbol E_s and the noise power spectral density N_0 , i.e.,

$$\frac{E_s}{N_0} = \frac{1}{\sigma_n^2}, \quad (2.6)$$

where σ_n^2 denotes the total variance of the complex noise sample. In simulations, this corresponds to setting the per-real-dimension variance as

$$\sigma_{n,r}^2 = \frac{1}{2 \cdot \text{SNR}}, \quad (2.7)$$

which reflects complex additive white Gaussian noise (AWGN) with independent real and imaginary parts of equal variance.

2.5 Multi-User Received Signal and Interference Modeling

In the multi-user scenario, the received signal at the base station includes both the desired user's contribution and multi-user interference. Let $\mathbf{h}_u = [h_{1,u}, h_{2,u}, \dots, h_{N_{\text{rx}},u}]^T$ denote the channel vector from user u to the N_{rx} antennas, and let $\mathbf{b}_u[k] = [b_{u,1}[k], b_{u,2}[k], \dots, b_{u,N_s}[k]]$ denote the transmitted symbol vector of user u at time k .

At symbol time k , the complex baseband received signal across all antennas is:

$$\mathbf{Y}[k] = \sum_{v=1}^{N_u} \mathbf{h}_v \cdot \mathbf{b}_v[k] + \mathbf{N}[k], \quad (2.8)$$

where $\mathbf{N}[k] \sim \mathcal{CN}(\mathbf{0}, \sigma_n^2 \mathbf{I})$ is the AWGN vector.

Focusing on a specific user u , the received signal can be separated as:

$$\mathbf{Y}_u[k] = \underbrace{\mathbf{h}_u \mathbf{b}_u[k]}_{\text{Desired signal}} + \underbrace{\sum_{\substack{v=1 \\ v \neq u}}^{N_u} \mathbf{h}_v \mathbf{b}_v[k]}_{\triangleq \mathbf{N}_u[k] \text{ (Interference + Noise)}} + \mathbf{N}[k], \quad (2.9)$$

where $\mathbf{N}_u[k]$ represents the total disturbance seen by user u at time k , composed of AWGN and multi-user interference.

The DL approach explored in subsequent chapters does not perform explicit interference cancellation; instead, the learning-based detector is trained to mitigate this effect directly from the generated data.

2.6 Low-Resolution Quantization

In this work, the focus is on using quantized (low-resolution) data to investigate detection robustness under severe information loss. Quantization is applied separately to the real and imaginary parts of each received entry. Two quantization methods were explored:

■ 1-bit Quantization:

$$Y_{r,k}^q = \text{sign}(\Re\{Y_{r,k}\}) + j \text{sign}(\Im\{Y_{r,k}\}). \quad (2.10)$$

■ Hyperbolic tangent Quantization:

$$Y_{r,k}^q = \tanh(\alpha \cdot \Re\{Y_{r,k}\}) + j \tanh(\alpha \cdot \Im\{Y_{r,k}\}), \quad (2.11)$$

where α controls the quantization sharpness.

Here, $r = 1, \dots, N_{\text{rx}}$ denotes the receive antenna index, and $k = 1, \dots, N_s$ represents the time (symbol) index. This coarse quantization dramatically simplifies hardware implementation but makes the detection problem significantly more challenging.

2.7 Correlation Matrix Construction and Feature Extraction

To extract meaningful and robust features from the quantized received signals for each user, block processing is performed to capture spatial correlation information while reducing noise effects. This step is essential to provide deep learning models with structured and informative inputs that improve detection performance. For each user u , the quantized block signal $\mathbf{Y}^q \in \mathbb{C}^{N_{\text{rx}} \times N_s}$ is processed to form a Hermitian correlation matrix, a common technique for capturing spatial dependencies in massive MIMO systems as shown in [MMF21].

$$\mathbf{Z}_u = \mathbf{Y}^{q,H} \mathbf{P}_u \mathbf{Y}^q, \quad (2.12)$$

where $\mathbf{P}_u = \text{diag}(p_{1,u}, p_{2,u}, \dots, p_{N_{\text{rx}},u})$ is the diagonal PSP weighting matrix. The Hermitian property arises naturally since $\mathbf{Z}_u$ is constructed via conjugate transpose.

To reduce dimensionality, the upper triangular part (excluding the diagonal) of $\mathbf{Z}_u$ is vectorized. Then all user vectors are concatenated:

$$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_u}]. \quad (2.13)$$

This procedure obtains a compact feature representation that is suitable for machine learning-based detection.

2.8 Label Preparation

The extracted feature vectors are paired with corresponding labels for supervised learning. For each time step k , the symbols of all users $\{a_u[k]\}$ are combined into a single integer index using the standard encoding approach described in [MMF21].

$$\text{Index} = \sum_{u=1}^{N_u} a_u[k] \cdot M^{N_u-u}. \quad (2.14)$$

These indices are subsequently transformed into one-hot vectors each of length M^{N_u} . These are binary vectors where only the element corresponding to the correct class is set to one while others are zero. This format is commonly used in classification tasks and will be further detailed in chapter 5, Neural Network Architecture for Non-Coherent Detection.

With this comprehensive system model, the next chapter is advancing to discuss the relevant background for the conventional non-coherent detection methods. This founda-

tional understanding will then set the stage for introducing the considered neural network architecture and detection strategy in subsequent chapters.

Chapter 3

Conventional Non-Coherent Detection Methods

This chapter reviews conventional approaches for non-coherent detection in massive MIMO systems. Two baseline techniques are discussed, decision-feedback differential detection (DFDD) and multiple-symbol differential detection (MSDD).

3.1 Decision-Feedback Differential Detection (DFDD)

Decision-feedback differential detection (DFDD) is a non-coherent detection technique designed to improve performance over simple differential detection while maintaining lower complexity than optimal multiple-symbol differential detection (MSDD). DFDD leverages a feedback structure to incorporate past decisions into the detection of current symbols, thus reducing error propagation. The subsequent explanation of DFDD is adapted from [YF20].

In DFDD, each symbol is detected individually, and the decisions are made successively using a feedback loop. The estimated symbol at time index k for user u is given by

$$b_{k,u}^{\text{DFDD}} = e^{j \text{MPSK}\{\sum_{l=0}^{k-1} b_{l,u}^{\text{DFDD}} z_{k,l,u}\}}, \quad (3.1)$$

where $\text{MPSK}\{\cdot\}$ denotes the quantization operator that maps its argument to the nearest M -PSK constellation point phase. It is defined as

$$\text{MPSK}\{x\} \triangleq \frac{2\pi}{M} \left\lfloor \frac{M}{2\pi} \arg(x) \right\rfloor. \quad (3.2)$$

To further improve detection performance, an optimized sorting order can be employed. In this case, the symbols are detected in an order that minimizes the accumulated phase

quantization error. Denote the optimized detection order indices by $\hat{k}_n$, then the sorted detected symbol is expressed as

$$b_{\hat{k}_n}^{\text{DFDD}} = e^{j\text{MPSK}\left\{\sum_{l=0}^{n-1} b_{\hat{k}_l}^{\text{DFDD}} z_{\hat{k}_n, \hat{k}_l, u}\right\}}. \quad (3.3)$$

The optimum sorting order is determined by selecting the next symbol to be detected as described in [YF20]:

$$\hat{k}_n = \arg \min_{\bar{k} \in \{0, \dots, N-1\} \setminus \{\hat{k}_0, \dots, \hat{k}_{n-1}\}} \left| \Delta_{\text{MPSK}} \left\{ \sum_{l=0}^{n-1} b_{\hat{k}_l}^{\text{DFDD}} z_{\bar{k}, \hat{k}_l, u} \right\} \right|, \quad (3.4)$$

where the phase quantization error is defined as

$$\Delta_{\text{MPSK}}\{x\} \triangleq \text{mod}_{s, 2\pi}(\arg(x) - \text{MPSK}\{x\}). \quad (3.5)$$

Here, $\text{mod}_{s, 2\pi}$ denotes the symmetrical modulo operator which reduces the argument into the interval $(-\pi, +\pi]$.

After detecting the differentially encoded symbols, the original information symbols can be recovered using

$$a_k = b_k b_{k-1}^*. \quad (3.6)$$

To mitigate multi-user interference in massive MIMO scenarios, successive interference cancellation (SIC) is integrated into DFDD. In each iteration, after estimating and removing the strongest user's contribution, the detection for the remaining users proceeds on a reduced interference signal. This integration of SIC with DFDD offers further robustness, especially under severe interference conditions.

Overall, DFDD provides a good trade-off between complexity and performance. While it does not achieve the optimal performance of MSDD (or its practical implementation via multiple-symbol differential sphere decoding, MSDSD), it significantly outperforms simple differential detection (DD) and does so with much lower computational effort compared to MSDD. By utilizing optimized decision order and feedback, DFDD remains a practical choice for large-scale massive MIMO systems operating in non-coherent regimes.

3.2 Multiple-Symbol Differential Detection (MSDD)

Multiple-symbol differential detection (MSDD) addresses the suboptimality of symbol-by-symbol detection by jointly considering a block of N_s symbols. Instead of relying solely on immediate past decisions, MSDD maximizes a metric over all possible symbol sequences

within the block. The following discussion of MSDD principles follows the treatment in [MMF21].

The optimal joint decision rule for user u is

$$\mathbf{b}_u^{\text{MSDD}} = \arg \max_{\bar{\mathbf{b}} \in \mathcal{M}^N, \bar{b}_0=1} \bar{\mathbf{b}} \mathbf{Z}_u \bar{\mathbf{b}}^H, \quad (3.7)$$

where $\mathcal{M}$ denotes the M -ary DPSK constellation set.

MSDD offers significant performance gains compared to DFDD and DD. However, the computational complexity grows exponentially with N_s since the detector needs to evaluate M^{N_s-1} possible sequences. To reduce complexity, the multiple-symbol differential sphere decoding (MSDSD) algorithm is often used. MSDSD reduces the search space by exploring only sequences within a sphere of a predefined radius, achieving near-optimal performance with feasible complexity.

In this thesis, MSDD and MSDSD are introduced as theoretical performance references. However, the machine learning-based approach is specifically compared with DFDD, since its simpler structure makes it a more realistic baseline in practical systems.

Chapter 4

Machine Learning Background and Tools

4.1 Machine Learning Classification for Non-Coherent Detection

Machine learning (ML) has emerged as a powerful tool for solving complex problems in wireless communications, especially when analytical model-based solutions become intractable or suboptimal under practical constraints. One prominent application is classification-based detection, where the goal is to assign each observed signal to one of several possible transmitted symbol classes. The following explanations are largely based on foundational material from [G125].

4.1.1 Fundamentals of Classification

At its core, classification in ML refers to the task of predicting a discrete label from input data. Given an input feature vector, the classifier maps it to a label chosen from a finite set of classes. In the context of signal detection, each possible transmitted symbol combination corresponds to a class.

The two main components in this setup are *features* and *labels*. Features represent the numerical or categorical information extracted from the input data that is relevant for distinguishing between classes. Labels indicate the correct class to which each feature vector belongs. During training, the model learns to approximate the mapping from features to labels.

4.1.2 Neural Network Architecture

A neural network is a popular model used for classification tasks due to its high flexibility and ability to approximate complex functions. It is composed of interconnected layers of units known as neurons.

- **Input layer:** Receives the feature vector. The dimension of this layer matches the number of extracted features.
- **Hidden layers:** Intermediate layers consisting of neurons that apply transformations to the input. The design of these layers (number, size, and activation functions) significantly affects performance.
- **Output layer:** Produces the final class probabilities. Its dimension equals the number of possible classes in the problem.

4.1.3 Activation Functions

Each neuron applies an activation function to introduce non-linearity, enabling the network to model complex relationships. Commonly used activation functions include:

- **ReLU (Rectified Linear Unit):** Defined as $f(x) = \max(0, x)$. It is widely used due to its simplicity and effectiveness in avoiding the vanishing gradient problem.
- **Softmax:** Used in the output layer of multi-class classification networks. It converts raw output values (logits) into probabilities that sum to one, allowing interpretation as confidence scores for each class.

4.1.4 Loss Functions and Optimization

During training, a loss function quantifies the difference between predicted outputs and true labels. For classification tasks, the most commonly used loss is *categorical cross-entropy*, defined as

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i), \quad (4.1)$$

where C is the number of classes, y_i is the true probability (typically 1 for the correct class and 0 otherwise), and $\hat{y}_i$ is the predicted probability.

The model parameters are updated iteratively to minimize the loss using optimization algorithms such as stochastic gradient descent (SGD) or Adam, an adaptive optimizer that adjusts learning rates for individual parameters.

4.1.5 Training Procedure

Training involves repeatedly presenting the model with batches of feature-label pairs and adjusting its weights to improve performance. Key terms include:

- **Epoch:** One complete pass over the entire training dataset.
- **Batch size:** Number of samples processed before updating the model weights.

The choice of epoch count and batch size influences convergence speed and generalization performance.

4.1.6 Evaluation Metrics

After training, the classifier is evaluated using metrics that reflect its predictive quality. Typical metrics for classification include:

- **Accuracy:** The proportion of correctly predicted samples over all predictions.
- **Precision and recall:** Particularly important when dealing with imbalanced classes.
- **Confusion matrix:** A matrix summarizing correct and incorrect predictions across all classes.

These metrics provide insight into the model's strengths and potential weaknesses, such as systematic misclassification of certain classes.

4.1.7 Types of Machine Learning and Motivation for Classification

Machine learning encompasses various paradigms depending on how models are trained and the type of tasks they solve. The most common categories include:

- **Supervised learning:** Models are trained on labeled data, where each input is paired with a known target output. Tasks include classification (assigning inputs to discrete classes) and regression (predicting continuous values).
- **Unsupervised learning:** Models learn patterns or structures from data without explicit labels. Typical tasks include clustering (grouping similar samples) and dimensionality reduction (finding compact representations).

- **Reinforcement learning:** Agents learn to take sequential actions in an environment to maximize a cumulative reward. This approach is widely used in control systems and decision-making problems.

In the context of signal detection, especially for non-coherent massive MIMO systems, the problem naturally fits into the supervised learning framework. The goal is to infer which symbol or symbol combination was transmitted based on received noisy and possibly quantized signals. Here, each possible transmitted symbol (or sequence of symbols) represents a discrete class, and the task is to classify each received block into one of these classes.

Unsupervised learning methods are less suitable because they do not leverage explicit ground-truth symbol information and instead focus on discovering unknown structures or groupings in the data. Reinforcement learning, while powerful for dynamic control problems, is not directly applicable to one-shot symbol decision tasks where immediate and precise decisions are required rather than sequential policies.

Machine learning-based classifiers, therefore, offer several advantages in communication systems. They can implicitly handle nonlinearities, adapt to channel variations without explicit parameter estimation, and mitigate interference by learning from large datasets. Moreover, data-driven approaches can outperform traditional detectors in scenarios where analytical models are insufficient or impractical.

Due to these reasons, this work focuses on a supervised classification approach for non-coherent signal detection. By formulating detection as a multi-class classification problem, the study can fully exploit the power of data-driven learning to achieve robust and accurate symbol recovery.

In summary, by leveraging the classification framework of machine learning, robust and scalable non-coherent detection strategies are enabled.

4.2 Tools and Libraries for Model Training

The supervised learning experiments in this work were implemented entirely in Python, leveraging a set of well-established scientific computing and machine learning libraries. Each tool played a distinct role in enabling an efficient workflow from dataset generation to model training, evaluation, and result visualization.

The core deep learning framework employed was *TensorFlow* with its integrated *Keras* high-level API [Ten25]. This environment provided both the computational backend for

large-scale numerical operations and the expressive interface for defining complex neural network architectures. The model was built using fully connected layers with **ReLU** activations and multiple output branches corresponding to each symbol decision. Training was conducted using the **Adam** optimizer, chosen for its adaptive learning rate mechanism that accelerates convergence in high-dimensional parameter spaces. The **categorical_crossentropy** loss function was applied to each output branch, reflecting the multi-class nature of the detection task. Hyperparameters such as the learning rate, number of epochs, and batch size were tuned to balance convergence speed and generalization performance.

Additionally, the Keras **Sequential** model class was used in some implementations to build neural networks in a straightforward, layer-by-layer fashion, providing simplicity for experimenting with network architectures. To handle large-scale datasets efficiently during training, the **Sequence** utility class from Keras was also employed. This class supports safe and efficient data generation and batching, enabling on-the-fly dataset creation without loading the entire dataset into memory. Detailed descriptions and applications of these tools are provided in the subsequent chapters.

Efficient numerical computation and array manipulation were handled by the *NumPy* library [Num25]. Its vectorized operations were essential for large-scale dataset preparation, including generation of random symbol sequences, channel realizations, and correlation matrix features. NumPy's broadcasting and indexing capabilities also streamlined intermediate processing steps, ensuring minimal overhead when working with millions of samples.

For data storage and exchange, the *SciPy* library [Sci25] was employed, specifically its **savemat** function for saving simulation outputs in MATLAB-compatible **.mat** format. This ensured compatibility with existing signal processing toolchains and facilitated further post-processing without conversion overhead.

Finally, performance visualization was carried out using *Matplotlib* [Mat25]. The generated plots, such as symbol error rate (SER) versus energy-per-symbol-to-noise ratio (E_s/N_0), were created in publication-quality format, using logarithmic scales where appropriate to highlight performance trends across a wide dynamic range.

Together, these libraries formed a cohesive and efficient computational ecosystem, allowing the proposed detection scheme to be implemented, trained, and evaluated at scale with both clarity and reproducibility.

With this background established, the following chapter proceeds to describe the proposed neural network architectures and detection schemes in detail.

Chapter 5

Neural Network Architecture for Non-Coherent Detection

In this chapter, two different neural network architectures are adopted from [MMF21], along with the consideration of different data generation and quantization methods.

5.1 Block Prediction Approach

The first neural network architecture explored from [MMF21] in this thesis follows a blockwise detection paradigm, where the model is trained to jointly predict the entire transmitted symbol sequence of length $N_s - 1$ in a single classification output. This approach treats the detection task as a multi-class classification problem, where each possible symbol sequence corresponds to one class label. For each input feature vector derived from the received quantized signal, the network attempts to classify the entire symbol block at once.

5.1.1 Output Structure and Label Encoding

For a system with M -ary modulation and a single user ($N_u = 1$), the total number of distinct symbol sequences of length $N_s - 1$ is given by

$$C = M^{N_s-1}.$$

Accordingly, the output layer of the network consists of C neurons, each representing a possible symbol sequence class. For example, using 4-DPSK modulation and a sequence length $N_s = 10$, this results in $M^9 = 262,144$ output classes — which quickly becomes infeasible in terms of training sample size, model complexity, and memory usage.

To generate class labels compatible with this format, each symbol sequence is represented by

$$\mathbf{a} = [a[1], a[2], \dots, a[N_s - 1]], \quad (5.1)$$

with symbols $a[k]$ taking values in $\{0, 1, \dots, M - 1\}$. The sequence is then encoded as an integer index:

$$\text{Index} = \sum_{k=1}^{N_s-1} a[k] \cdot M^{N_s-1-k}. \quad (5.2)$$

This index is then converted to a one-hot encoded vector of length C , forming the target label used during training.

While this encoding approach is conceptually straightforward and mathematically exact, it leads to severe scalability limitations. The exponential growth in the number of output classes with respect to N_s makes training practically infeasible for moderate or large sequence lengths.

5.1.2 Network Structure

The network architecture used in this approach consists of a standard feed-forward neural network with the same configuration as in [MMF21]:

- **Input layer:** Accepts feature vectors extracted from the correlation matrices $\mathbf{Z}_u$ of quantized signals. The input dimension is fixed and identical across all architectures.
- **Hidden layers:**
 - First hidden layer: Fully connected, 128 neurons, ReLU activation.
 - Second hidden layer: Fully connected, 64 neurons, ReLU activation.
- **Output layer:** Fully connected, M^{N_s-1} neurons, softmax activation.

The ReLU (Rectified Linear Unit) activation function is chosen for the hidden layers due to its non-saturating gradient and empirical success in training deep networks. The output layer uses the softmax function to produce a valid probability distribution over all possible symbol sequences. Figure 5.1 serves as a visual for the block prediction approach.

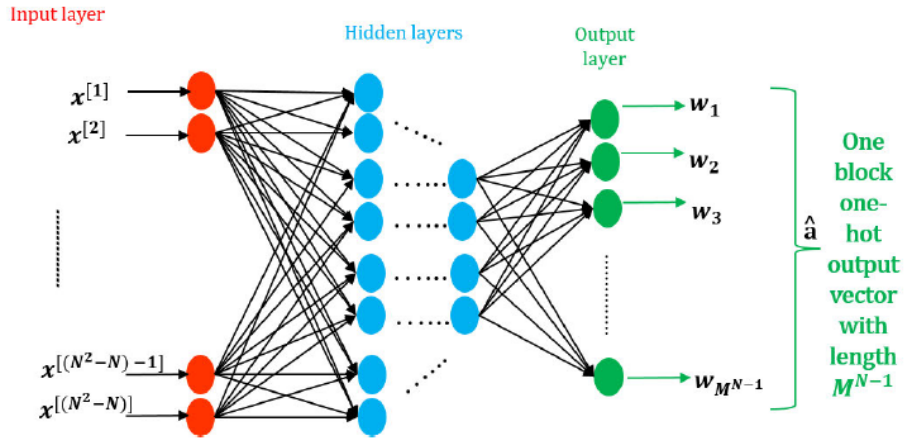


Figure 5.1: Illustration of the architecture model where one Dense layer at the output is responsible for predicting the whole information block. Taken from [MMF21].

5.1.3 Loss Function and Optimization

The network is trained using the categorical cross-entropy loss function:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i),$$

where y_i is the true class label (one-hot encoded), and $\hat{y}_i$ is the predicted probability for class i .

Optimization is carried out using the Adam optimizer [Ten25] with a learning rate of 0.001. Adam is widely used in deep learning due to its adaptive moment estimation, which combines the benefits of RMSProp and momentum and is well-suited for sparse gradients and noisy objectives like those found in communication systems.

The network is trained with:

- **Batch size:** 64
- **Epochs:** 20

5.1.4 Limitations and Motivation for a New Architecture

Although this architecture performs reasonably well for small sequence lengths (e.g., $N_s = 3$ or 5), it becomes highly impractical for longer sequences due to:

- Exponentially large output dimension.
- Requirement of enormous training datasets to populate all classes.
- Increased overfitting risk due to sparse class representation.
- Memory limitations on modern GPUs and even high-spec CPUs.

For example, assuming 1×10^6 training samples, the network would only be able to cover a tiny fraction of the 2.6×10^5 possible classes when $N_s = 10$ and $M = 4$, resulting in poor generalization and convergence issues.

These scalability concerns motivated the development of a more practical and modular architecture — discussed next — which addresses these challenges while maintaining detection accuracy.

5.2 Per-Symbol Parallel Prediction

To overcome the scalability limitations of the blockwise prediction model, a new architecture was developed in [MMF21] that adopts a *per-symbol prediction* approach. In this model, the network no longer predicts the entire sequence as a single classification output. Instead, it predicts each symbol in the sequence independently but in parallel, enabling it to scale efficiently with longer sequences and support multi-user detection.

5.2.1 Output Structure and Label Encoding

In this architecture, each time step $k = 1, \dots, N_s - 1$ is assigned its own dedicated output layer. For every user and each time step, the transmitted symbols are jointly represented as a class label.

For the **single-user case** ($N_u = 1$), each output consists of:

M output neurons, one per symbol class.

For the **multi-user case** ($N_u > 1$), the class label encodes the joint symbol values of all users at a given time. The total number of classes becomes:

$$C = M^{N_u}.$$

For instance, with 3 users and 4-DPSK modulation, the number of possible joint symbol combinations is $4^3 = 64$, leading to 64 output neurons per time step.

The encoding of labels in this case is done by collapsing the vector of user symbols into a single integer class index using

$$\text{Index} = \sum_{u=1}^{N_u} a_u[k] \cdot M^{N_u-u}, \quad (5.3)$$

where $a_u[k] \in \{0, 1, \dots, M-1\}$ is the symbol transmitted by user u at time k . This index is then transformed into a one-hot vector of dimension M^{N_u} , forming the target label for that symbol time step.

This encoding is vastly more efficient than the previous architecture's:

$$\text{Old: } C = M^{N_s-1} \quad \text{vs} \quad \text{New: } C = M^{N_u} \text{ (per symbol)}. \quad (5.4)$$

Since $N_s - 1 \gg N_u$ in most practical systems, the new formulation significantly reduces the output dimensionality, memory requirements, and training complexity.

5.2.2 Network Architecture

The improved architecture retains the same input layer and hidden layers as the previous model, maintaining consistency in feature extraction and network depth. However, the output structure is changed to consist of multiple output heads — one per symbol prediction. All ML model parameters were taken from [MMF21].

- **Input layer:** Same as before — the concatenated feature vector derived from all users' correlation matrices $\mathbf{Z}_u$.
- **Hidden layers:**
 - Dense layer with 1000 neurons and ReLU activation.
 - Dense layer with 200 neurons and ReLU activation.
- **Output layers:**
 - $N_s - 1$ parallel dense layers.
 - Each has M^{N_u} neurons and uses softmax activation.

The use of multiple output layers enables simultaneous prediction of all symbols in the block, without the need to predict the entire sequence jointly. This significantly improves training efficiency and memory usage while retaining full block output. Figures 5.2 and

5.3 visually illustrate the structure of the per-symbol parallel prediction approach and its compatibility with both single user and multiple users scenarios.

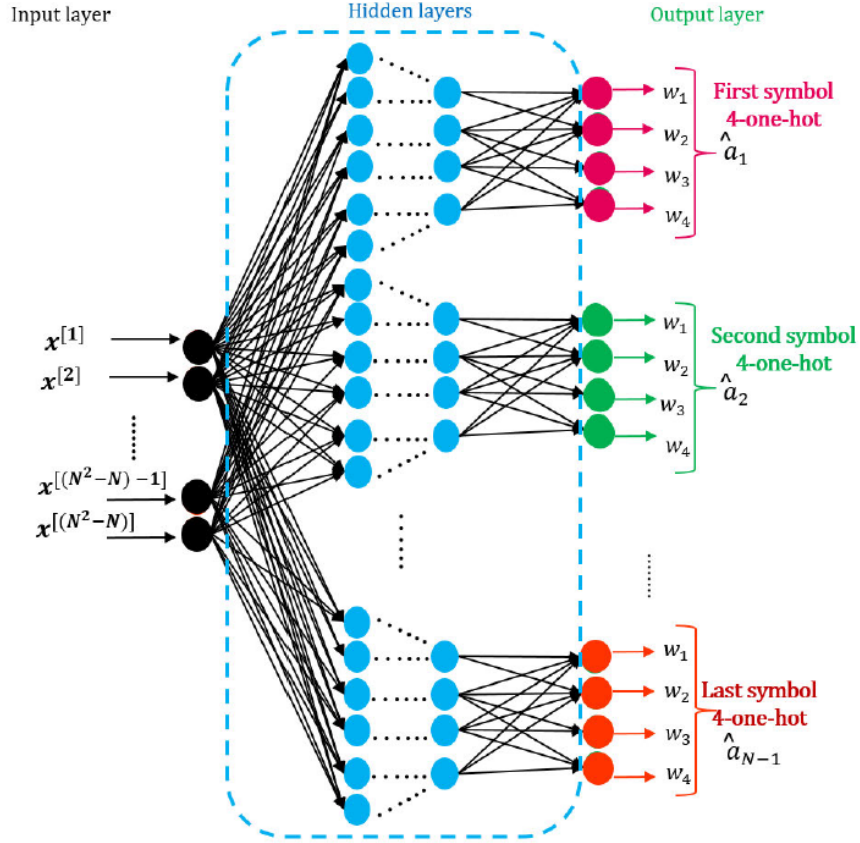


Figure 5.2: Illustration of the new architecture model in the single user scenario where $N_s - 1$ dense layers at the output with M neurons each are responsible for predicting their respective information symbol. Taken from [MMF21].

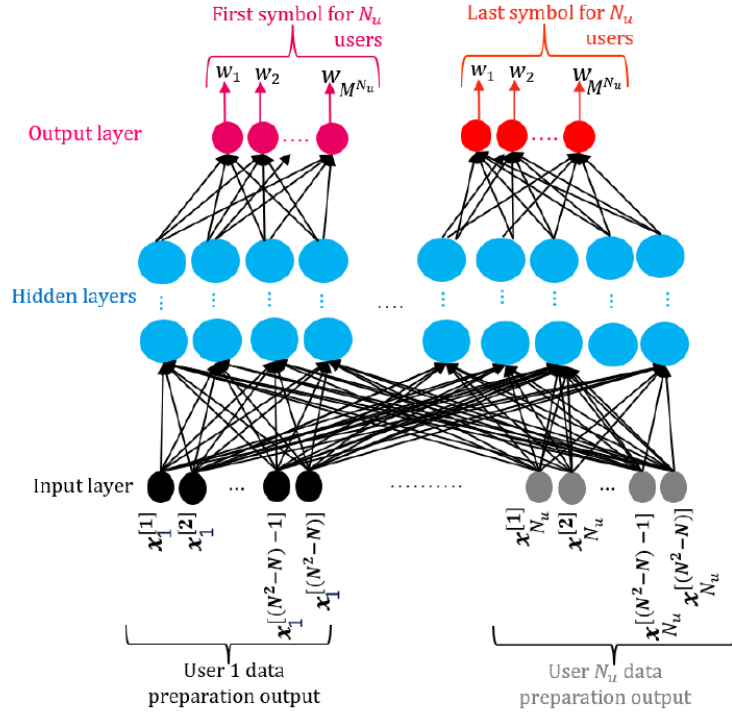


Figure 5.3: Illustration of the new architecture model in the multiple users scenario where $N_s - 1$ dense layers at the output with M^{N_u} neurons each are responsible for predicting their respective information symbol. Taken from [MMF21].

5.2.3 Loss Function and Optimization

Each output layer is treated as an independent classification problem with categorical cross-entropy loss

$$L_k = - \sum_{i=1}^{M^{N_u}} y_{k,i} \log(\hat{y}_{k,i}),$$

where $y_{k,i}$ is the true one-hot label for time step k and $\hat{y}_{k,i}$ is the predicted probability for class i . The total loss is the sum of all individual symbol losses:

$$L = \sum_{s=1}^{N_s-1} L_{k,s}.$$

The network is optimized using the Adam optimizer with:

- Learning rate: 0.0001
- Epochs: 10

- Batch size: 32

Adam is particularly suitable for communication systems due to its adaptive step size, efficient convergence, and robustness to noisy or sparse gradients that naturally arise in symbol detection under quantized signals.

5.2.4 Input Features and Consistency Across Architectures

Both the first and second architectures use the same input layer and preprocessing pipeline. The input vector is built by computing user-specific correlation matrices from quantized received signals

$$\mathbf{Z}_u = \mathbf{Y}^{\mathbf{q},\mathbf{H}} \mathbf{P}_u \mathbf{Y}^{\mathbf{q}}, \quad (5.5)$$

followed by extracting the upper triangular (excluding diagonal) elements and concatenating all user features into one input vector

$$\mathbf{x} = [\text{vec}_{\text{upper}}(\mathbf{Z}_1), \text{vec}_{\text{upper}}(\mathbf{Z}_2), \dots, \text{vec}_{\text{upper}}(\mathbf{Z}_{N_u})],$$

where $\text{vec}_{\text{upper}}$ represents the extraction of the upper triangle elements of the correlation matrices. This consistent design ensures that the two models can be fairly compared while exploring the impact of output formulation and loss structure.

5.2.5 Advantages of Per-Symbol Architecture

The following advantages of the per-symbol architecture are discussed in [MMF21] and motivate their use in this work.

- **Scalability:** Avoids exponential blow-up in output dimensionality with longer sequences.
- **Modularity:** Easily extended to multi-user detection by increasing M^{N_u} output size per symbol.
- **Efficiency:** Enables training on practical hardware even for large sequence lengths.
- **Parallelism:** Facilitates simultaneous prediction of all symbols, improving throughput and simplifying label generation.

This architecture forms the backbone of the proposed solution and is used in all subsequent experiments and evaluations throughout this thesis.

5.3 Data Generation Techniques

A critical component in training machine learning-based detectors is the generation of representative and diverse training data. In this work, two distinct data generation strategies were explored: one based on fixed, pre-generated datasets, and the other based on dynamic, on-the-fly generation during training. Each approach offers different trade-offs in terms of memory usage, training efficiency, and generalization capabilities.

5.3.1 Pre-Generated Dataset (Offline Sequence Generation)

In the first approach, a large dataset of training samples is generated and stored in memory prior to training. Each sample includes:

- The received quantized signal block,
- Its corresponding feature vector, and
- The ground-truth symbol labels for all time steps.

Once generated, this dataset is reused for every training epoch, allowing the model to see the same samples multiple times. For example, if the dataset contains one million unique samples and training is done over ten epochs, the model will iterate over the same one million samples ten times.

This technique allows for efficient use of processing time during training since data is only generated once. It also enables better memorization, as the model has the opportunity to refine its predictions on the same examples across multiple epochs. However, it requires substantial memory resources to store large datasets, particularly when dealing with long sequences, many users, or large feature vectors.

5.3.2 On-the-Fly Generation (Sequential Data Generation During Training)

In contrast, the second approach generates new data at the beginning of every training epoch. Instead of reusing the same samples, each epoch presents the model with entirely new, randomly generated sequences. Over ten epochs, the model would encounter ten million different training samples if one million are generated per epoch.

This method is far more memory efficient, as it avoids the need to store large datasets. It also encourages generalization by exposing the model to a wider variety of channel

realizations, interference patterns, and noise levels. However, a key limitation of this technique is that the model does not see any sample more than once. As a result, it becomes more difficult for the model to consolidate what it learns from one batch to the next, especially in tasks where a degree of memorization is beneficial.

5.3.3 Summary of Trade-Offs

Table 5.1 shows a comparison between the two data generation strategies.

Table 5.1: Comparison of the two training data generation strategies.

Aspect	Pre-Generated Dataset	On-the-Fly Generation
Sample diversity per epoch	Low (fixed samples reused)	High (new samples every epoch)
Memory usage	High (stores all data)	Low (generates as needed)
Learning behavior	Favors memorization	Favors generalization
Computational cost	Lower (no generation overhead)	Higher (generation during training)
Risk of overfitting	Higher (fixed dataset)	Lower (randomized input)

Both methods were used and studied in this work. The pre-generated strategy supports better stability during training and enables fine-tuning of the model’s parameters over a familiar dataset. The on-the-fly generation strategy introduces increased randomness and variability, helping to mimic realistic deployment conditions. The choice between these techniques depends on the target system’s constraints and whether priority is placed on consistency or adaptability.

5.4 Quantized Data and Quantization Using Hyperbolic Tangent

In modern massive MIMO systems, receiver designs must account for low-resolution analog-to-digital converters (ADCs), which motivate the use of quantized data in detec-

tion architectures. This work considers several quantization techniques, including 1-bit quantization and quantization based on the hyperbolic tangent ($\tanh$) function.

5.4.1 1-Bit Quantization

In 1-bit quantization, each complex entry of the received signal is coarsely quantized by applying a sign function to both its real and imaginary parts:

$$Y^q = \text{sign}(\Re\{Y\}) + j \cdot \text{sign}(\Im\{Y\}). \quad (5.6)$$

This harsh quantization retains only the sign information and is useful for reducing hardware complexity. However, it removes fine-grained amplitude information that could be helpful for detection, especially in low SNR regimes.

5.4.2 Quantization Using Hyperbolic Tangent

Inspired by the work in [KS20], an approximation of 1-bit quantization is adopted using the hyperbolic tangent function

$$Y^q = \tanh(\alpha \cdot \Re\{Y\}) + j \cdot \tanh(\alpha \cdot \Im\{Y\}), \quad (5.7)$$

where α controls the steepness of the $\tanh$ function. For small values of α , the quantizer is smooth and less aggressive, while for large α , the behavior approximates that of a hard 1-bit quantizer.

The quantizer with tunable α was originally introduced in [KS20] in the context of model-based deep learning for one-bit compressive sensing. Their work demonstrated that progressive learning with increasing quantization sharpness improves convergence and accuracy in extreme quantization regimes. Figure 5.4 illustrates the transformation of the hyperbolic tangent curve as a result of increasing α over training.

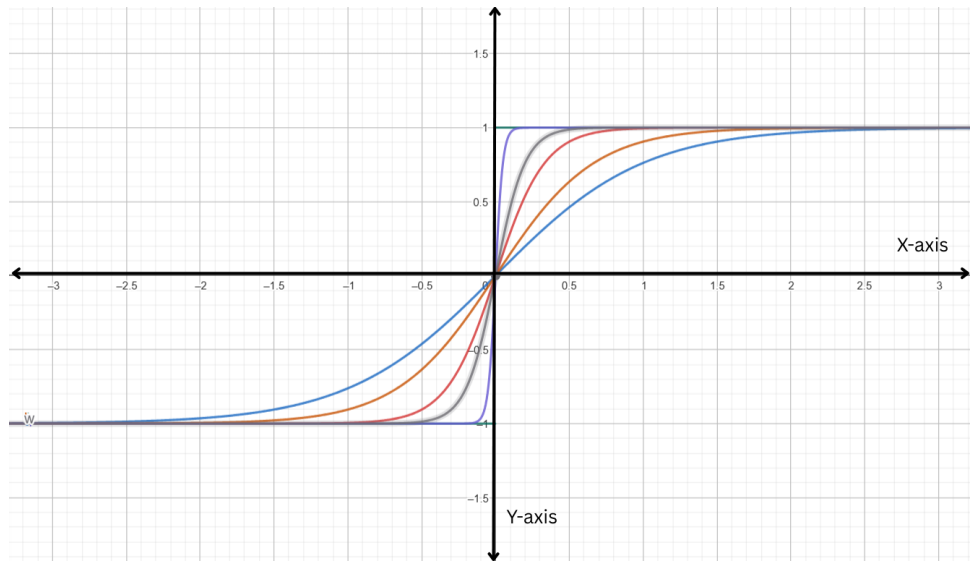


Figure 5.4: Illustration of the evolution of the tanh function throughout the training procedure till it acts as a 1-bit quantizer.

5.4.3 Integration into Learning-Based Detection

The quantizer is incorporated into the data generation pipeline to simulate the receiver's behavior. Several strategies are considered:

- **Static Quantization:** A fixed α is used throughout training, providing a smooth approximation to 1-bit quantization.
- **Step-wise Increasing α :** The value of α is increased after each training step, progressively hardening the quantization.
- **Epoch-wise Increasing α :** The quantization sharpness is updated at the end of each epoch.

These variants are inspired by [KS20] and allow the neural network to adapt to the increasing difficulty of the detection task, gradually learning to operate under near 1-bit conditions. By training on progressively sharpened quantized data, the model avoids instability at the start of training and benefits from a curriculum learning effect.

5.4.4 Application and Justification

The use of tanh-based quantization offers two major advantages according to [KS20]:

1. **Gradient Flow:** Unlike the discontinuous sign function used in hard quantization, $\tanh$ is differentiable, making it more suitable for backpropagation and gradient-based optimization.
2. **Curriculum Learning Strategy:** Starting with hyperbolic tangent quantization allows the model to learn basic structures before gradually handling more aggressive quantization scenarios.

These hyperbolic tangent quantization techniques form an essential part of the training methodology for deep learning-based detection under quantized observations.

Chapter 6

SER Performance Evaluation

This chapter presents the Symbol Error Rate (SER) performance results for the proposed deep learning-based multiple-symbol differential detector (DL-MSDD) compared to the conventional method decision feedback differential detection (DFDD) under various simulation settings, following the presentation style in [MMF21]. The training phase is performed using a random signal-to-noise ratio (SNR) sampled uniformly from the range 5 dB to 19 dB, with 2 dB increments. In contrast, the testing phase uses newly generated data, where each SNR point is evaluated separately using one million samples per SNR value. The training and testing phases are fully decoupled to ensure proper generalization and avoid overfitting. For quantized simulations, although different quantization techniques are used in training (1-bit, tanh, increasing α), all evaluations are performed on 1-bit quantized data. The only exception is the non-quantized training models, which are also tested on non-quantized data.

The power space profile (PSP) for each user is modeled as a Gaussian distribution over the antenna array with a spread parameter $\zeta^2 = 400$. The number of receive antennas is $N_{\text{rx}} = 100$, and in the multi-user scenario, three single-antenna users are considered.

The SER metric is calculated using the decoded class indices of both predicted and true labels. The predicted output probabilities are converted into class indices using the $\arg \max$ operation and then mapped back into per-user symbol vectors via the inverse index-to-symbol mapping

$$a_u[k] = \left\lfloor \frac{\text{Index mod } M^{N_u-u+1}}{M^{N_u-u}} \right\rfloor, \quad (6.1)$$

where $a_u[k]$ denotes the decoded symbol for user u at symbol time k . The SER is calculated by counting mismatches per user across all detected symbols. Mathematically, this process follows

$$\text{SER}_u = \frac{1}{N_{\text{total}}} \sum_{k=1}^{N_s-1} \sum_{i=1}^{N_{\text{samples}}} \mathbf{1} \{ \hat{a}_u^{(i)}[k] \neq a_u^{(i)}[k] \}, \quad (6.2)$$

where $\hat{a}_u^{(i)}[k]$ is the predicted symbol for user u at symbol time k in sample i , and N_{total} is the total number of symbols across all users and time steps.

6.1 Simulation Environment and Model Details

Table 6.1 indicates a summary for key parameters and setups for the conducted simulations.

Table 6.1: Simulation Setup Summary

Parameter	Value
Framework	TensorFlow (Keras API)
Programming Language	Python
Processors Used	Intel Xeon E7-4850 v2 @ 2.30 GHz AMD Ryzen 9 5950X @ 3.4 GHz
System RAM	125 GB and 251 GB
Fading Channel Model	Flat Rayleigh fading with AWGN
Modulation Order	$M = 4$ (4-DPSK)
Users	3 single-antenna users
Receive Antennas	$N_{\text{rx}} = 100$
User PSP Spread	$\zeta^2 = 400$
Learning Rate	0.0001
Activation Functions	ReLU (hidden layers), Softmax (output)
Output Layer Shape	M for single user, M^{N_u} for multi-user

6.2 SER Performance Graphs and Results

6.2.1 Single User – Non-Quantized

In this subsection, the focus is on non-quantized datasets for both DL-MSDD and DFDD.

For the first graph in Figure 6.1, the DL-MSDD simulation was conducted using the pre-generated dataset approach, where all training data was generated prior to the training process. The dataset size was 50 million samples, and the model was trained for 10 epochs. It can be observed that the DL model outperforms DFDD for the first four SNR points. At the fifth SNR point, the performance becomes nearly identical, with DFDD slightly better. For higher SNR values, particularly in the last three points, the DL model's performance degrades, while DFDD reaches the optimal solution ($SER = 0$) at the highest SNR.

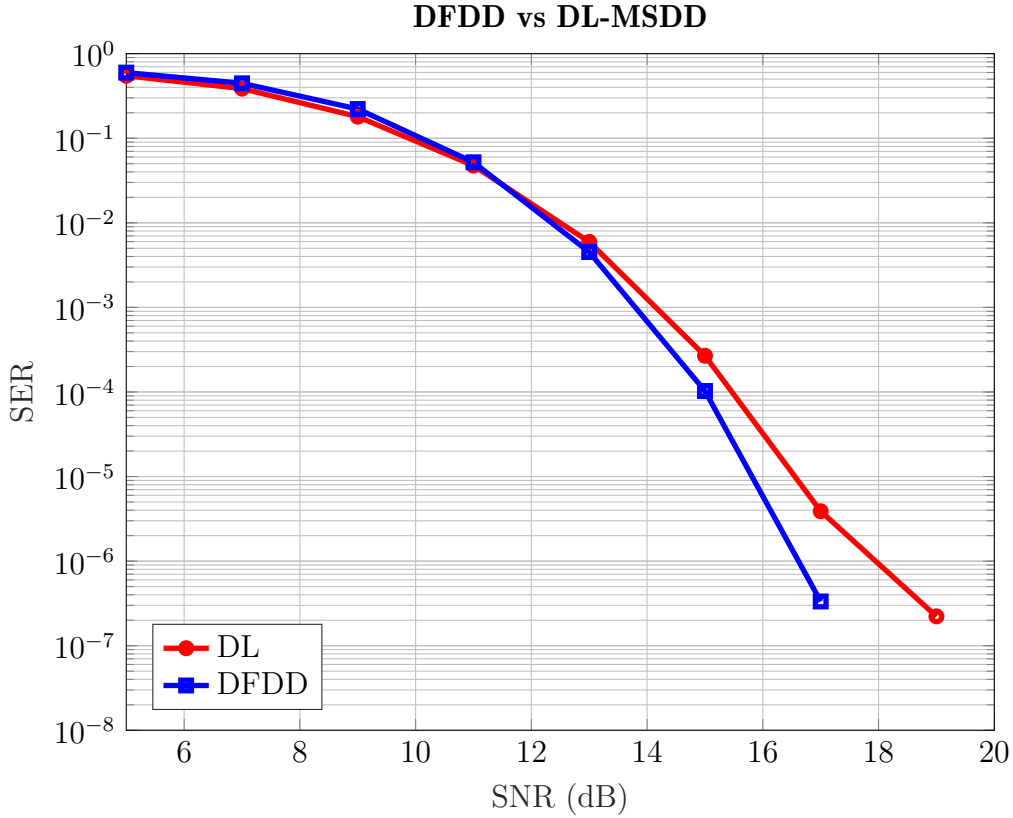


Figure 6.1: SER performance for DL-MSDD pre-train generator single user non-quantized data with 50 million samples, 10 epochs

For the second graph in Figure 6.2, the DL-MSDD simulation employed the *on-the-fly* data generation approach, where smaller datasets are generated dynamically at each training step. In this case, the dataset size was 100 million samples, and the model was trained for 15 epochs. The results show that DFDD surpasses the DL model from the fourth SNR point onward, indicating that the pre-generated data approach used in Figure 6.1 achieved better overall performance in this scenario.

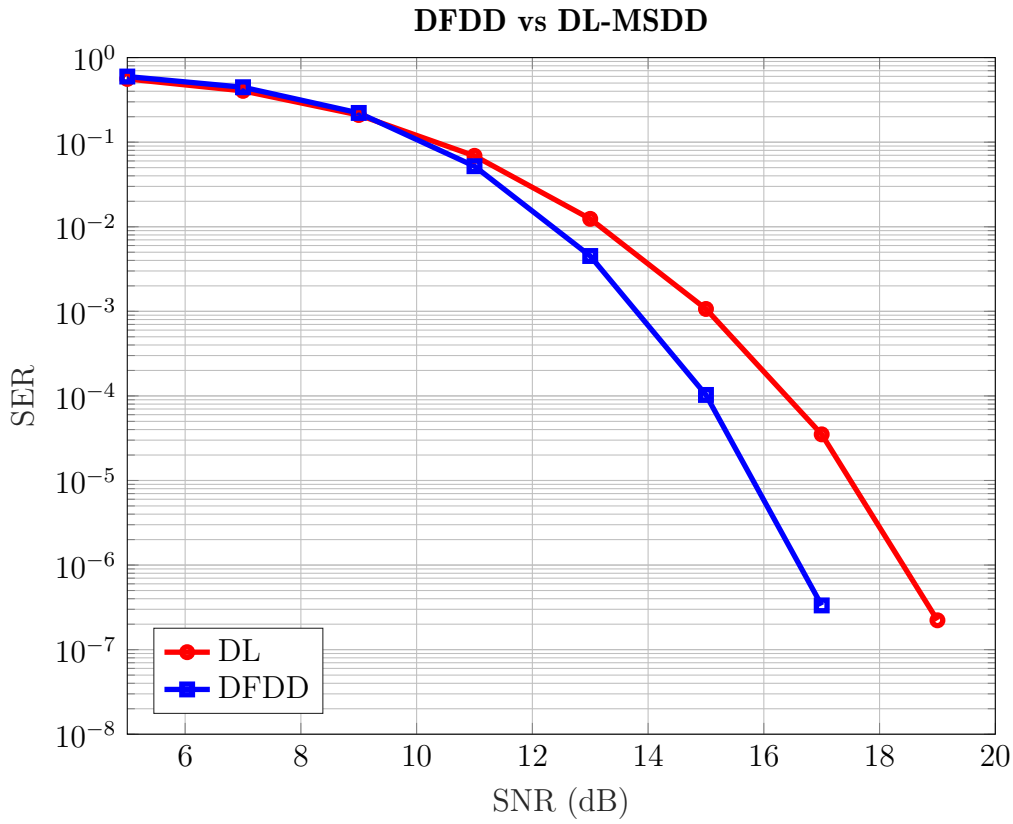


Figure 6.2: SER performance for DL-MSDD on-the-fly generator single user non-quantized data with 100 million samples, 15 epochs

6.2.2 Single User – 1-bit Quantized Training

In this subsection, the focus shifts to low-resolution (quantized) datasets for both DL-MSDD and DFDD. The DL model is trained on a 1-bit quantized dataset, and both detection techniques are evaluated using 1-bit quantized test data.

The first three graphs, shown in Figures 6.3, 6.4, and 6.5, present simulation results obtained using the pre-generated dataset method, with sample sizes of 25 million, 30 million, and 50 million, respectively, and the number of epochs for all three simulations is 10. In all three cases, the DL model outperforms DFDD for the first five SNR points, whereas DFDD achieves better performance in the last three SNR points. Additionally, a slight improvement in the convergence of the SER curve can be observed as the sample size increases from 25 to 50 million samples across the three figures.

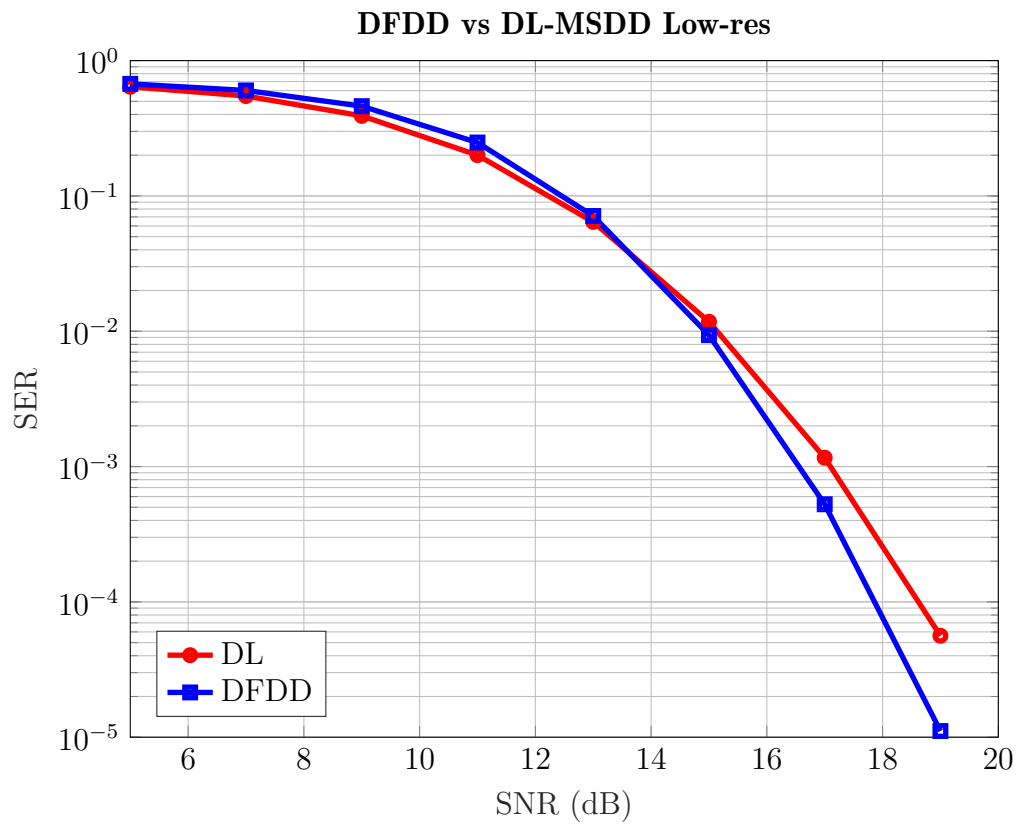


Figure 6.3: SER performance for DL-MSDD pre-train generator single user 1-bit quantized data with 25 million samples, 10 epochs

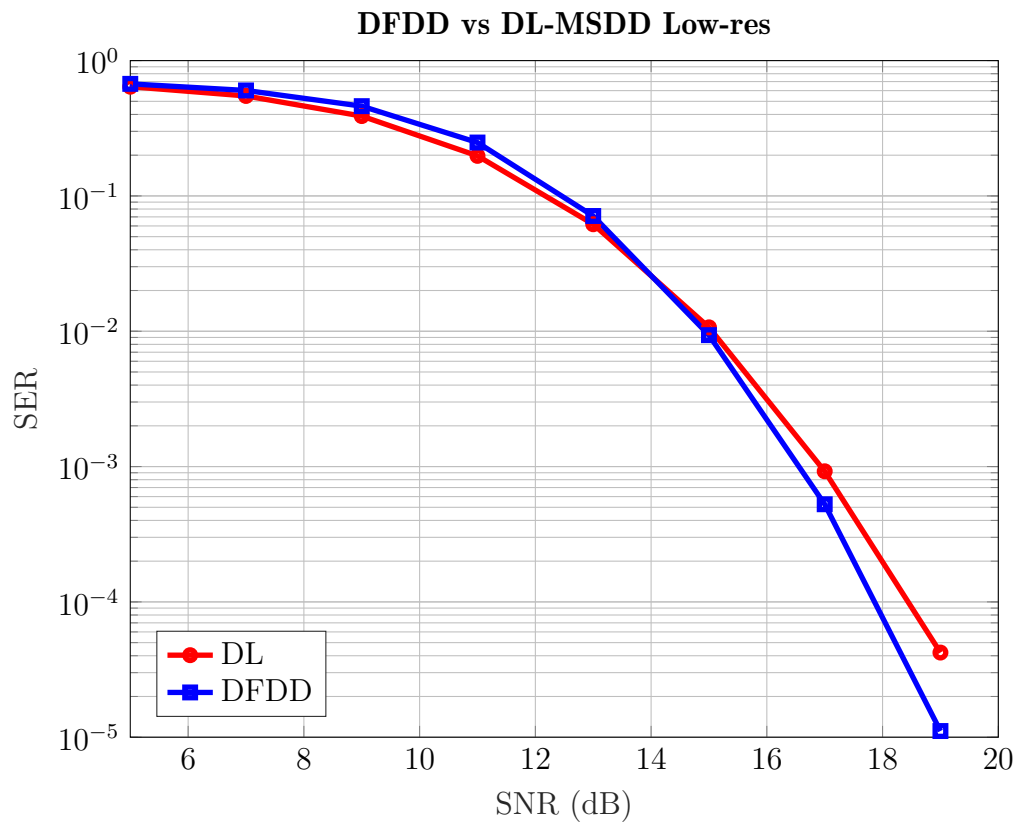


Figure 6.4: SER performance for DL-MSDD pre-train generator single user 1-bit quantized data with 30 million samples, 10 epochs

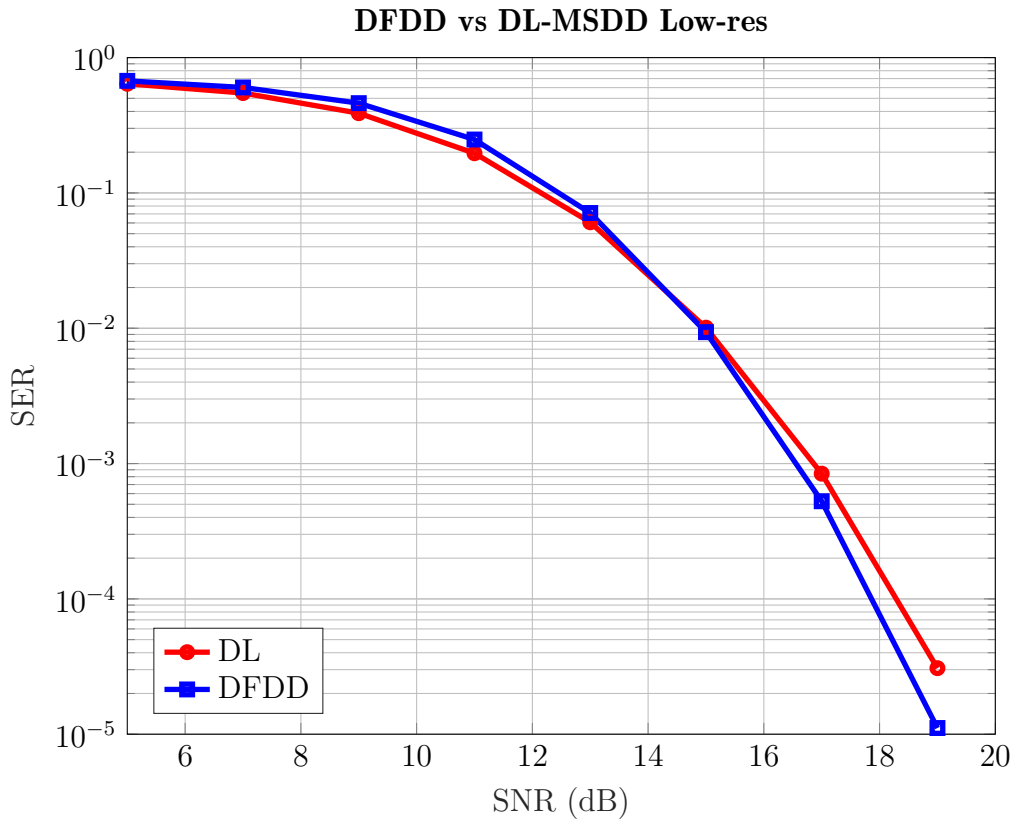


Figure 6.5: SER performance for DL-MSDD pre-train generator single user 1-bit quantized data with 50 million samples, 10 epochs

Figure 6.6 examines the *on-the-fly* data generation technique applied to low-resolution data. The sample size for this simulation is 100 million, where the model trained for 15 epochs. The results indicate that the DL model outperforms DFDD in low SNR conditions, particularly across the first four points. However, from the fifth point onward, DFDD demonstrates superior performance. This suggests that the simulations using the pre-generated data technique in Figures 6.3, 6.4, and 6.5 achieve better overall SER performance.

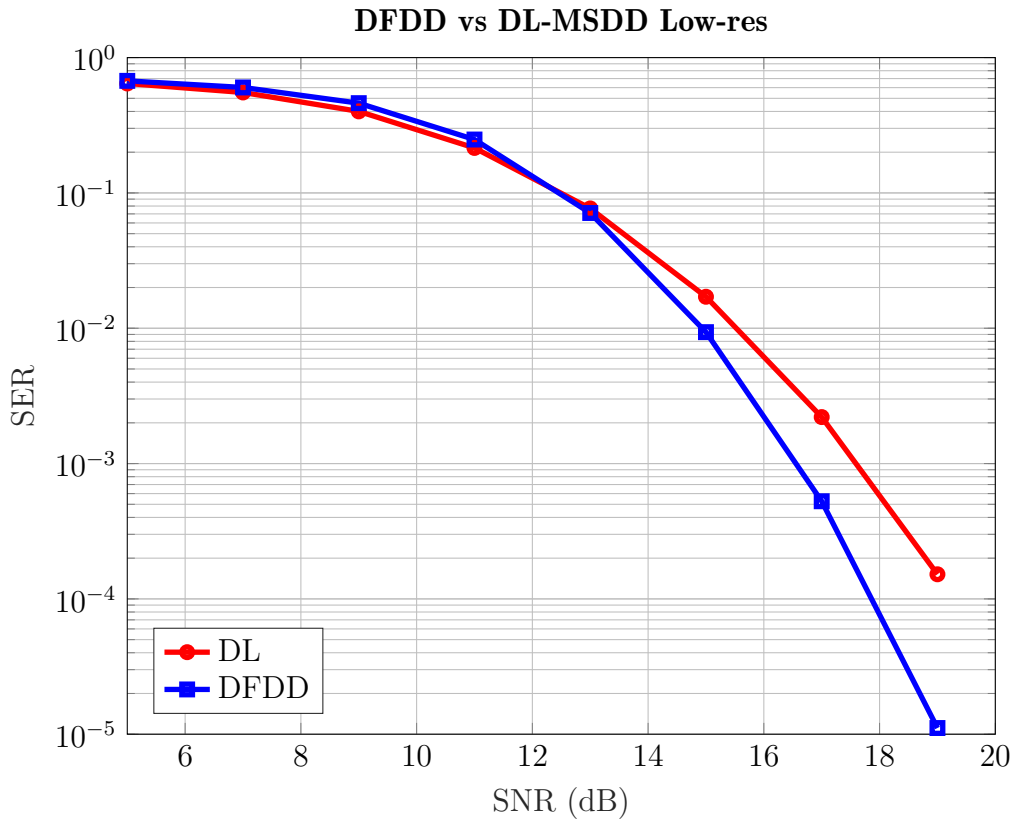


Figure 6.6: *SER performance for DL-MSDD on-the-fly generator single user 1-bit quantized data with 100 million samples, 15 epochs*

6.2.3 Single User – Trained with Hyperbolic Tangent Quantizer

In this subsection, the focus remains on quantized datasets, but a different quantization technique is applied during the DL model's training phase. Instead of the 1-bit quantizer, the hyperbolic tangent function is used in the DL-MSDD training procedure. However, both DL-MSDD and DFDD are ultimately evaluated using 1-bit quantized datasets.

Three variations of tanh-based quantization training are evaluated.

In Figure 6.7, the scaler α is fixed to 1 throughout the training procedure. The model uses *on the fly* generation method with a 25 million sample size and is trained for 15 epochs. This simulation as observed results in a significantly degraded SER across all SNR values, making the model incomparable with DFDD.

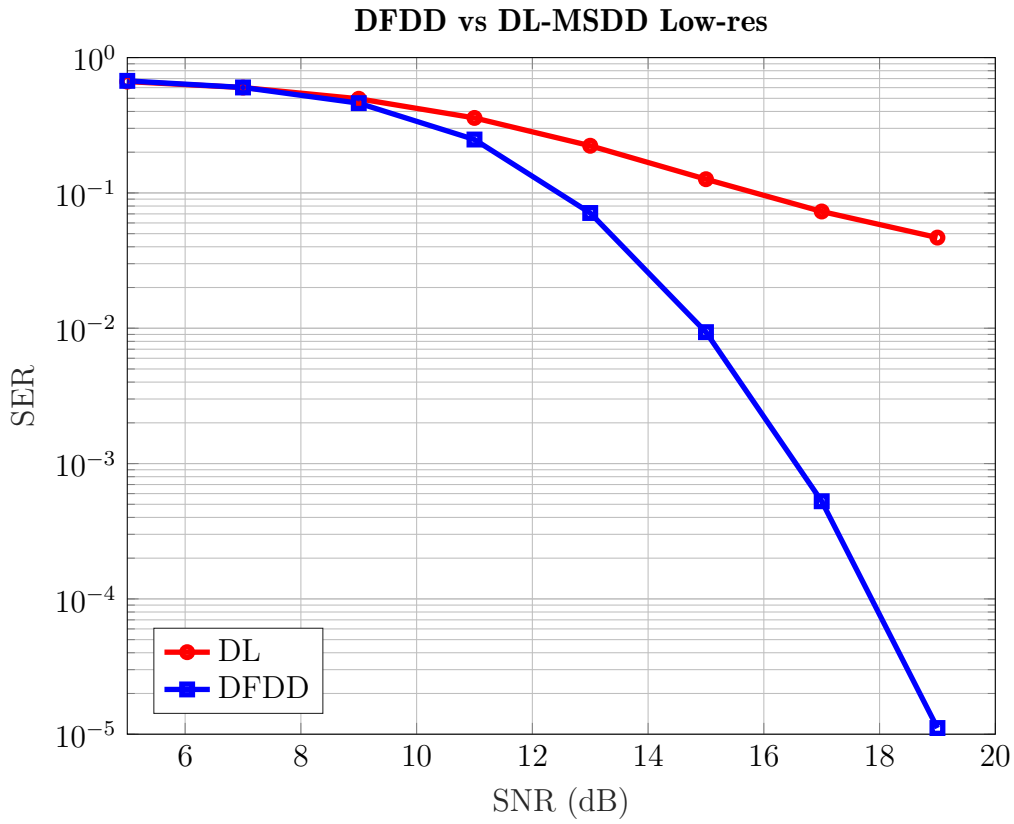


Figure 6.7: *SER performance for DL-MSDD on-the-fly generator single user tanh quantized data ($\alpha = 1$ fixed through training) with 25 million samples, 15 epochs*

In contrast, Figure 6.8 shows competitive SER trends very similar to the 1-bit quantized *on the fly* model shown in Figure 6.6. The scaler α is increasing per training step. The model also uses the *on the fly* generation method with a 25 million sample size and is trained for 15 epochs. Pre-train generation remains advantageous in overall performance.

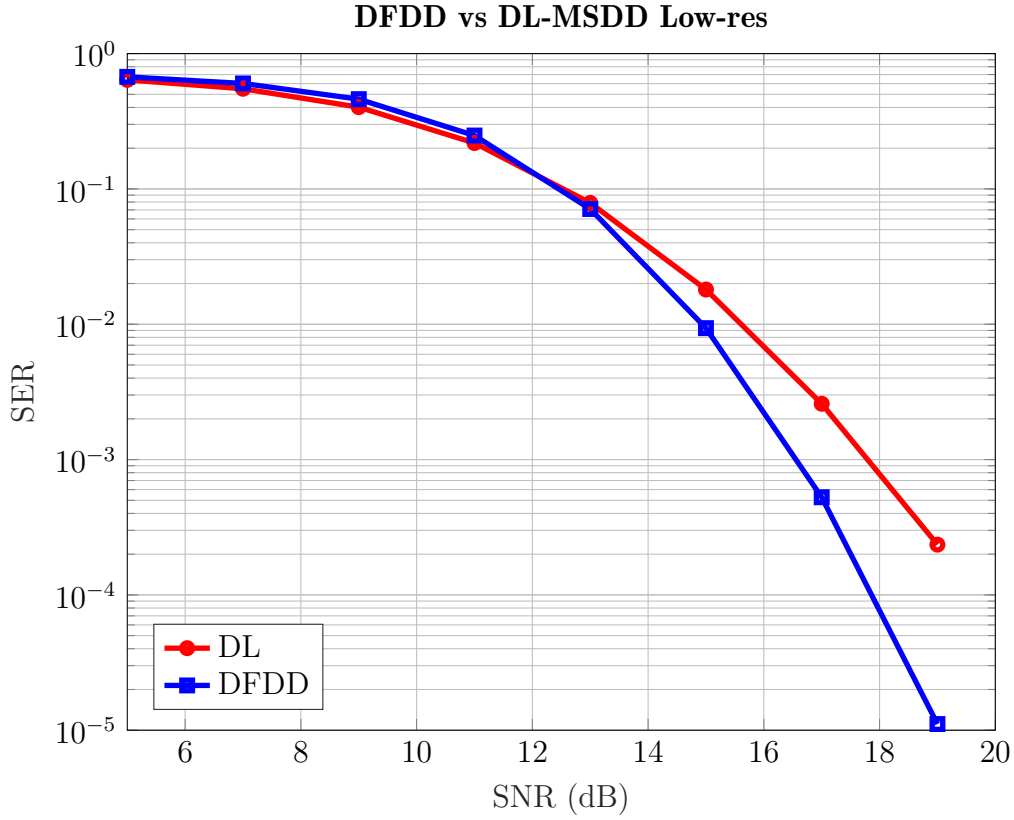


Figure 6.8: *SER performance for DL-MSDD on-the-fly generator single user tanh quantized data ($\alpha = 1 \rightarrow 20$ increased per step/epoch while training) with 25 million samples, 15 epochs*

As for Figure 6.9, the results are nearly identical to Figure 6.8. Here α is increased per epoch and the model uses *on the fly* generation as all the hyperbolic tangent based quantization simulations. The sample size is 100 million and the model trained for 15 epochs. As observed, 1-bit quantizer pre-train generation technique shown in Figure 6.5 has the best performance in comparison to other techniques.

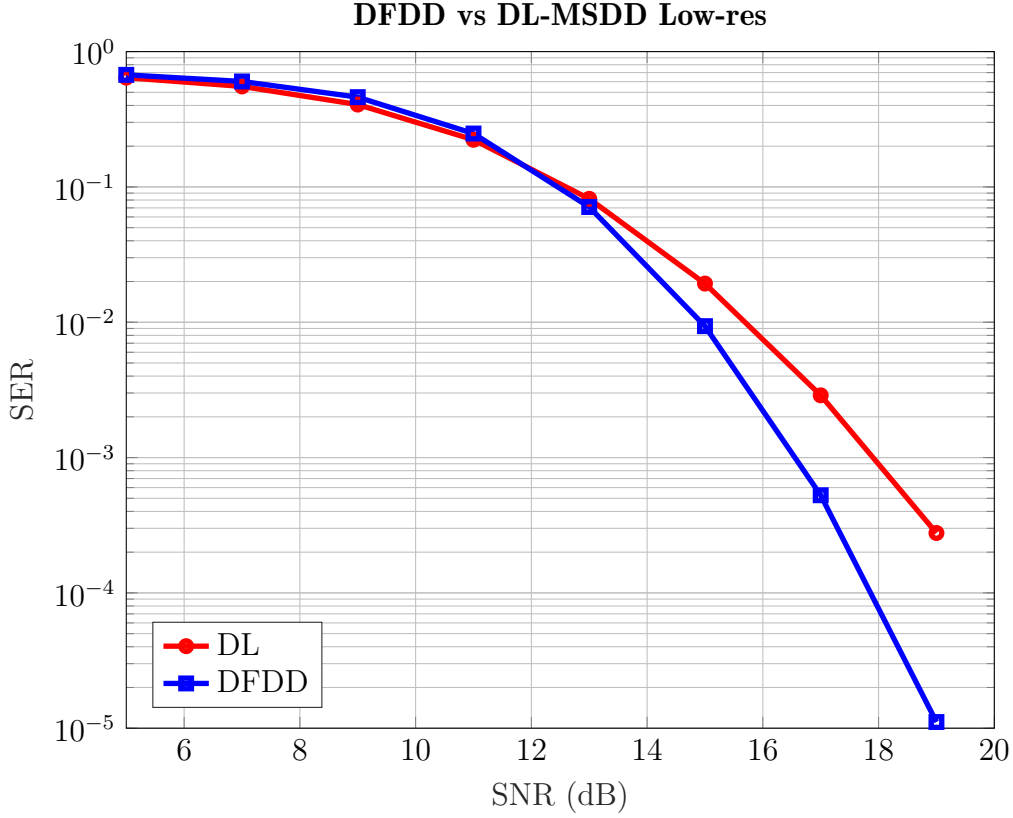


Figure 6.9: *SER performance for DL-MSDD on-the-fly generator single user tanh quantized data ($\alpha = 1 \rightarrow 20$ increased per epoch while training) with 100 million samples, 15 epochs*

6.2.4 Multi-User Scenario – 1-bit Quantized Training

The multi-user results under 1-bit quantization are shown for both pre-train and on-the-fly generation in Figures 6.10 and 6.11. The sample size for the pre-train model is 15 million samples, while the on-the-fly model has a 25 million sample size, both models trained for 25 epochs. Both methods yield similar performance across most SNR points. DL-MSDD performs better in all SNR points across all users except the last three SNR points for user 3 only. Users 1, 2, and 3 experience different levels of interference based on their peak antenna centers at indices [20, 50, 85], respectively. User 2, positioned between the others, suffers the highest interference. User 3, being the least affected, performs better at higher SNRs.

The superior performance of DL-MSDD can be attributed to its ability to implicitly learn and mitigate multi-user interference from the received data. By capturing spatial correlations and patterns in the quantized signals, the deep learning model adapts to the different interference levels experienced by each user, allowing more robust detection

compared to conventional methods (DFDD).

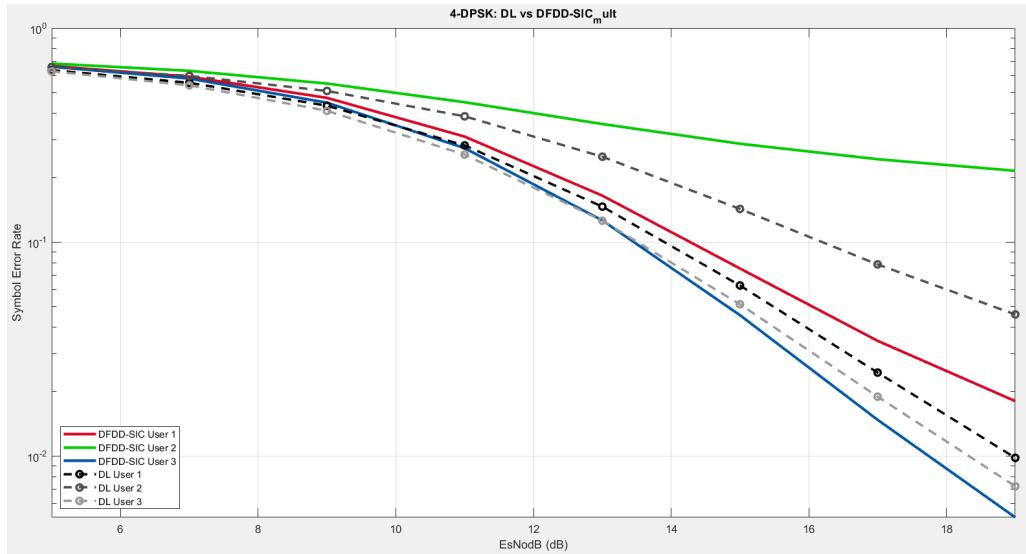


Figure 6.10: *SER performance for DL-MSDD pre-train generator 3 users 1-bit quantized data with 15 million samples, 25 epochs*

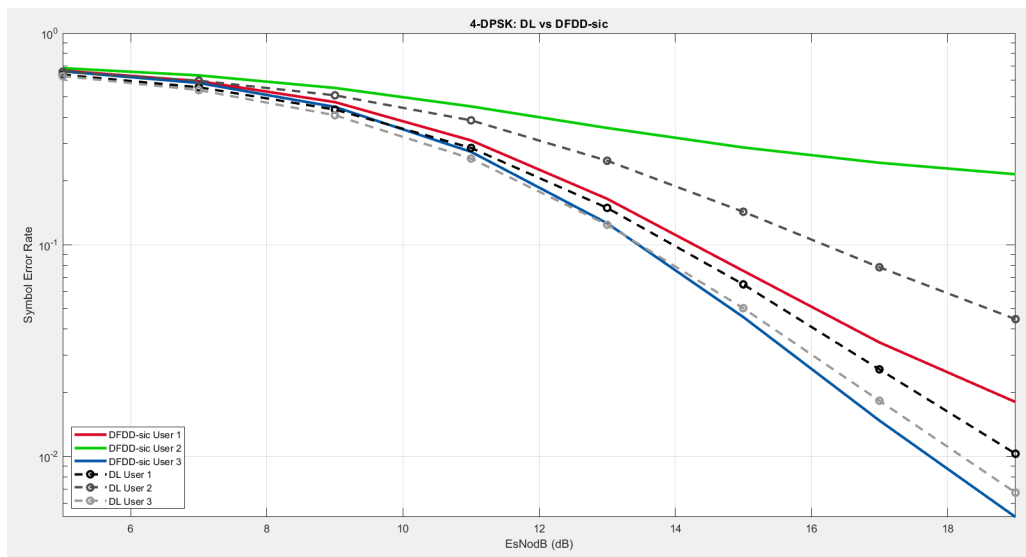


Figure 6.11: *SER performance for DL-MSDD on-the-fly generator 3 users 1-bit quantized data with 25 million samples, 25 epochs*

6.3 Conclusion

The SER performance evaluation across various simulation settings reveals several key insights. First, for non-quantized data, the pre-generated dataset approach consistently

outperforms the *on-the-fly* generation method, particularly at higher SNR values, where DFDD can match or surpass DL-MSDD. Under 1-bit quantization, DL-MSDD generally shows superior performance in the low-to-medium SNR range, while DFDD regains the advantage at higher SNRs. Increasing the sample size in pre-generated datasets leads to marginal but noticeable improvements in SER convergence.

For the tanh-based quantization training, a fixed scaling factor α severely degrades performance, whereas dynamically increasing α —either per step or per epoch—recovers performance to a level comparable with 1-bit quantized on-the-fly models. Nevertheless, across all quantized scenarios, pre-generated data with 1-bit quantization remains the most effective approach.

In the multi-user 1-bit quantized case, DL-MSDD maintains an advantage for most SNR points across users, though performance varies with interference levels determined by user positions in the antenna array. Overall, the results demonstrate that both the choice of quantization strategy and the data generation method play a critical role in achieving robust detection performance, particularly when operating under severe resolution constraints.

To highlight the main difference between the results presented in this chapter and those in [MMF21], the focus here is on the 1-bit low-resolution evaluation of the DL detection model, combined with a comparison between pre-generated and *on-the-fly* data generation techniques, as well as the exploration of a tanh-based quantization method. In contrast, [MMF21] primarily investigated non-quantized data using only the pre-generated data approach, comparing it with conventional methods such as DFDD and MSDSD.

Chapter 7

Summary and Outlook

7.1 Summary

This work investigated a deep learning-based receiver design for non-coherent massive MIMO systems with quantized data. A supervised learning framework was formulated to address the detection of differentially encoded DPSK signals under realistic hardware constraints, including 1-bit quantization and multi-user interference. Two neural network architectures were explored: a sequence-level predictor with a large output dimensionality, and a more scalable symbol-level parallel prediction architecture. The latter proved more efficient in terms of memory and practical feasibility, especially for long sequences and multi-user scenarios.

Two distinct data generation strategies were examined for training: offline (pre-train) generation and online (on-the-fly) generation. The offline strategy allowed the network to repeatedly learn from a fixed dataset, while the online strategy exposed the model to a wider variety of signal instances across training epochs.

The study also implemented several quantization techniques during training. A quantization layer based on the hyperbolic tangent function (`tanh`) was adopted from [KS20], and integrated using static, step-wise, and epoch-wise increasing sharpness parameters to approximate 1-bit quantization. All trained models were tested on either unquantized data or true 1-bit quantized data, depending on the experiment goal.

Performance was evaluated in terms of symbol error rate (SER) across a range of SNR values. Results demonstrated the benefit of deep learning at low SNR regimes and under interference, particularly in multi-user cases. In high-SNR single-user scenarios, traditional DFDD techniques remained strong competitors. The overall findings highlight the feasibility of data-driven detection methods in quantized and complex MIMO environments.

7.2 Outlook

Several directions remain open for future research:

- **Adaptive Quantization Strategies:** The sharpness parameter α in the tanh quantizer was increased manually. Future work could explore learnable or adaptive scheduling policies to better balance generalization and convergence during training.
- **Understanding On-the-Fly Generation Limitations:** Investigate the underlying causes for the observed performance gap between on-the-fly and pre-generated data approaches, despite the former's exposure to diverse datasets across epochs, to determine whether factors such as data distribution shifts, training instability, or batch statistics contribute to reduced detection accuracy.
- **Higher-Order Modulation:** The current system utilized $M = 4$ DPSK. Extending the framework to higher modulation orders could expose further insights into the trade-offs between complexity, quantization, and detection performance.
- **Joint User Detection and Interference Mitigation:** Although the current model implicitly handled interference, designing architectures that explicitly separate or mitigate user interference (e.g., via attention mechanisms or user-specific decoders) may enhance multi-user scalability.
- **End-to-End Learning:** Integrating the neural detector with trainable front-end components (e.g., trainable quantizers or soft antenna selection) could improve system robustness, particularly under dynamic or imperfect channel assumptions.
- **Hardware Deployment and Real-World Datasets:** Applying the developed models to real-world measurements or deploying them on hardware platforms with constrained computational resources would help bridge the gap between simulation and practice.

The results of this thesis contribute to the ongoing exploration of deep learning in communication systems and demonstrate its effectiveness in non-coherent, low-resolution massive MIMO environments. The modular nature of the system allows for continued extension and refinement in future work.

References

- [MMF21] O. Mahmoud, A. El-Mahdy and R. F. H. Fischer: *On Multi-User Deep-Learning-Based Non-Coherent DPSK Multiple-Symbol Differential Detection in Massive MIMO Systems*, IEEE Access, vol. 9, pp. 148339–148353, Nov. 2021, doi: 10.1109/ACCESS.2021.3124863.
- [KS20] S. Khobahi and M. Soltanalian: *Model-Based Deep Learning for One-Bit Compressive Sensing*, IEEE Transactions on signal processing, vol. 68, pp. 1053–587X, Jul. 2020, doi: 10.1109/TSP.2020.3022319.
- [YF20] G. Yammine and R. F. H. Fischer: *Decision-Feedback Differential Detection with Optimum Detection Order Metric for Noncoherent Massive MIMO Systems* IEEE Xplore, Universität Ulm, Institut für Nachrichtentechnik, 2020
- [FH23] Prof. Dr.-Ing. Robert Fischer and Prof. Dr.-Ing. Dr. h.c. Johannes Huber: *Digital Communications Lecture Chapter 5*, Universität Ulm, Institut für Nachrichtentechnik, April 2023.
- [Num25] NumPy library. <https://numpy.org/>
- [Mat25] Matplotlib library. <https://matplotlib.org/>
- [Ten25] TensorFlow library (Keras API). <https://www.tensorflow.org/>
- [Sci25] SciPy library. <https://scipy.org/>
- [Gl25] Web page of the Machine Learning Crash Course, Google for Developers. <https://developers.google.com/machine-learning/crash-course>
- [NT25] Web page of the Institute of Communications Engineering, Ulm University. www.uni-ulm.de/in/nt/